

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Student Result Processing System"

SUBMITTED BY:

Student Name: Mahi Mahi
Program: B.Tech
Branch: B.Tech(Computer Science and Engineering)

SUBMITTED TO:

IILM University, Greater Noida

Internship Details:

AICTE Student ID: STU6997f6d107cc01771566801
APAAR ID: 781587671015
Internship Duration: 10 Weeks (Apr 2026 - Jun 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Mahi Mahi**, a student of **IILM University, Greater Noida**, has successfully completed **10-Week (Apr 2026 - Jun 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**Student Result Processing System**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagar Bala Behera

Mentor, EduSkills Academy

Date: 16-07-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Mahi Mahi

(Signature)

Date: 16-07-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagar Bala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Computer Science and Engineering)** for their encouragement and support.

ABSTRACT

This project details the development of a Student Result Processing System designed to manage and analyze student academic performance. It progresses from basic data parsing to sophisticated calculations of percentages, letter grades, and final rankings, providing a comprehensive tool for academic record management.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **10 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The Student Result Processing System is a Python-based application engineered to streamline the management and analysis of student academic records. The system is capable of ingesting raw student performance data, performing essential calculations such as summing subject scores and determining percentage scores, and assigning standardized letter grades. Furthermore, it implements a ranking mechanism to order students based on their overall academic achievements. This project demonstrates practical application of core Python programming concepts for data manipulation and analytical tasks relevant to educational institutions.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The project was developed using a step-by-step approach. Initially, raw student data strings were parsed into structured lists and dictionaries. Subsequently, subject scores were summed to derive total marks for each student. Performance metrics, specifically percentages, were calculated based on these totals. A conditional logic system was implemented to assign letter grades based on performance scores. Finally, students were ranked according to their overall performance, enabling a clear overview of academic achievement.

6. Tools & Technologies Used

Programming Language: Python. Concepts: String manipulation, List processing, Dictionary management, Conditional logic (if-elif-else), Functions, JSON serialization.

7. Implementation & Results

Problem 1: Parsing Student Records

Description: The initial task involves processing raw student data, which is often provided in a delimited string format. This requires robust string manipulation techniques to split the data into individual components such as student names and subject scores, enabling further processing.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    if not line.strip():
        continue

    # Split the line by comma
    parts = line.split(",")

    # Extract name and convert marks to integers
    name = parts[0]
    marks = [int(mark) for mark in parts[1:]]

    # Store in the required dictionary format
    result = {
        "name": name,
        "marks": marks
    }

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 2: Summing Subject Scores

Description: This problem focuses on aggregating the individual subject scores for each student. Utilizing data structures like dictionaries, the system needs to efficiently sum up all recorded marks to provide a consolidated total score for each student.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    if not line.strip():
        continue

    data = json.loads(line)

    # Calculate sum of marks
    total = sum(data["marks"])

    # Add it to the dictionary
    data["total"] = total

    results.append(data)

print(json.dumps(results, separators=(',', ':')))
```

Problem 3: Performance Metrics

Description: The objective is to calculate key performance indicators from the aggregated scores. This specifically involves computing the percentage score for each student based on their total marks and the number of subjects, providing a standardized measure of academic performance.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    if not line.strip():
        continue

    data = json.loads(line)

    # Calculate percentage (total / 3)
    data["percentage"] = data["total"] / 3

    results.append(data)

print(json.dumps(results, separators=(',', ':')))
```

Problem 4: Grade Assignment

Description: Based on the calculated performance metrics (percentages), this task requires the implementation of a grading system. Conditional logic is employed to assign a letter grade (e.g., A, B, C) to each student according to predefined score thresholds.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    if not line.strip():
        continue

    data = json.loads(line)

    # Determine grade based on percentage
    if data["percentage"] >= 90:
        data["grade"] = "A"
    elif data["percentage"] >= 80:
        data["grade"] = "B"
    elif data["percentage"] >= 70:
        data["grade"] = "C"
    else:
        data["grade"] = "F"

    results.append(data)

print(json.dumps(results, separators=(',', ':')))
```

Problem 5: Final Ranking

Description: The final step is to establish an academic hierarchy among students. This involves sorting the students based on their overall performance metrics (e.g., total score or percentage), typically using their calculated grades or scores, to generate a final ranking list.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

all_students = []
for line in lines:
    if not line.strip():
        continue
    data = json.loads(line)
    all_students.append(data)

# Sort by 'total' in descending order
all_students.sort(key=lambda x: x["total"], reverse=True)

results = all_students

print(json.dumps(results, separators=(',', ':')))
```

8. Learning Outcomes

- Gained practical experience in data parsing and manipulation using Python's string and list methods.
- Developed proficiency in using dictionaries for organizing and accessing student data.
- Enhanced understanding of conditional logic for implementing grading systems.
- Learned to apply sorting algorithms for ranking students based on performance metrics.
- Acquired foundational knowledge in JSON serialization for data persistence.

9. Challenges Faced & Solutions

Challenge: Handling inconsistent data formats in raw input strings.

Solution: Implemented robust error checking and data cleaning routines during the parsing phase, along with flexible splitting logic to accommodate minor variations.

Challenge: Efficiently sorting a list of student dictionaries based on multiple criteria (e.g., total score, then name for ties).

Solution: Utilized Python's `sorted()` function with a lambda function as the key to specify the primary and secondary sorting order, ensuring accurate ranking.

Challenge: Designing a flexible grading scale that can be easily modified.

Solution: Defined grading thresholds in a separate configuration or dictionary, allowing for straightforward updates without altering core logic.

10. Conclusion

The Student Result Processing System successfully demonstrates the application of fundamental Python programming concepts to solve a real-world data management problem. By automating the processing, calculation, and ranking of student results, the system offers an efficient and reliable method for tracking academic performance.

11. Future Scope

- Extend the system to support different weighting schemes for subjects.
- Integrate with a database for persistent storage of student records.
- Develop a graphical user interface (GUI) for a more user-friendly interaction.
- Implement features for generating detailed individual student reports.

12. References

- Python Documentation: Official guide to Python language features and standard library.
- Python Tutorial: Comprehensive resources for learning Python programming.
- Stack Overflow: Community-driven platform for programming Q&A and problem-solving.

13. Annexure

- Internship Completion Certificate
- Offer Letter